Streaming Twitter Analytics with Kafka, Spark, and GCP

Angel Coronel

Project Presentation

CSc 84030 Big Data Analytics

12/5/25

Motivation

- Social media acts as a live sensor for public opinion
- Most opinions come from trendy or “important” people
  - Chose billionaires as point of interest (Taylor Swift, Rihanna, Elon Musk, Jeff Bezos) and the companies they might be tied to (Tesla, Amazon)
- Want to connect billionaires to each other based on what people say about them
  - Companies they are related to
  - Who gets mentioned with whom
  - What is the sentiment analysis of what is said (AI will be the judge)
- Twitter/X has a lot of continuous noise which necessitates scalable, streaming analytics
- Want to turn that raw stream into a structured, sentiment-aware relationship graph

Objectives

- Given a stream of tweets with billionaires and their companies in mind, want to:
  - Build a graph with people/companies as nodes and tweet mentions as edges
  - Add sentiment analysis to the weight of the edges
- Use a cloud streaming pipeline running on the cloud
  - Google Cloud hosts Kafka for ingestion of tweets, Spark streaming for processing the tweets and Parquet/Google Cloud Storage for storage, analysis & fault tolerance
- Add metrics to the project to understand results & the pipeline

Challenges

- Volume & Velocity
  - Need to process continuous Tweets in real time without falling behind
- Noise
  - Tweets are messy: languages, emojis, slang, spam
- Fault Tolerance
  - System must tolerate failures (e.g., node or driver crashes) without losing data
- Separation of Concerns
  - Ingestion, processing, and storage should be decoupled so each can scale independently
- Money
  - Twitter API has a very limited free developer account and GCP/GCS cost money to run

Pro

Unleash the full potential of X's v2 API

Apps	**3 environments**
DMs	**Over 300K requests per month per user**
Posts	**Retrieve up to 1M Posts per month**
Realtime Posts stream	**Access to realtime posts stream**
Users	**Over 8M requests per month per user**

SAVE 10%

$4500/month

$54000 billed annually

Upgrade now

$5000/month

$5000 billed monthly

Upgrade now

Basic

Begin your journey with X's v2 API

Apps	**2 environments**
DMs	**75K requests per month per user**
Posts	**Retrieve up to 15K Posts per month**
Realtime Posts stream	**No access**
Users	**50k requests per month per user**

SAVE 12.5% - LIMITED TIME

$175/month

$2100 billed annually

Upgrade now

$200/month

$200 billed monthly

Upgrade now

Free

Get limited access to X's v2 API

Apps	**1 environment**
Posts	**Retrieve up to 100 Posts and 500 writes per month**

You have access to this plan

Downgrade to free access?

Downgrade

GET /2/tweets/search/stream	50 requests / 15 mins **PER APP**	✗	✗
GET /2/tweets/search/stream/rules	450 requests / 15 mins **PER APP**	✗	✗

Tech Stack

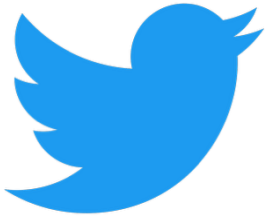




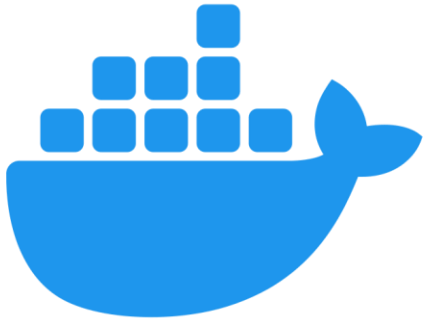
kafka

- Kafka
  - Distributed commit log for streaming data
  - Decouples producers (Twitter / replay script) from consumers (Spark, future apps)
- Spark Structured Streaming
  - Unified batch + streaming engine with DataFrame / SQL API
  - Built-in Kafka connector → simple to subscribe to topics
- Google Cloud Platform (GCP)
  - Compute Engine VMs & Google Cloud Storage (GCS)
- Docker & Docker Compose
  - Package Kafka, Spark, and dependencies in containers
    - Zookeeper
    - Kafka broker
    - Spark master + workers
- Python
- VADER
  - AI sentiment analysis tool
- Twitter API









System Architecture

- Data Source – Twitter stream
  - Producer normalizes tweets into JSON schema
- Ingestion Layer – Kafka
  - Producer publishes to Kafka topic tweets and Kafka acts as a log buffering between source and processing
- Streaming Processing – Spark Structured Streaming
  - Spark job reads from Kafka using the Kafka connector
  - Steps:
    - Parse JSON tweets into DataFrames
    - Match mentions to a YAML entity list (people & companies)
    - Apply VADER sentiment analysis
    - Emit raw graph edges (src, dst, sentiment, timestamp, tweet_id)
  - Write edges as Parquet files to Google Cloud Storage
- Batch Analytics Layer
  - Separate Spark/PySpark job reads Parquet edges from GCS
  - Form edges by (src, dst) into counts, avg sentiment, first/last seen

Data Model: Entities & Tweets

Tweets normalized by producer.

Figures represent people and companies.

```python
normalized = {
    "id": raw_tweet.get("id"),
    "created_at": raw_tweet.get("created_at"),
    "text": raw_tweet.get("text"),
    "author_id": raw_tweet.get("author_id"),
    "in_reply_to_user_id": None,
    "referenced_tweets": raw_tweet.get("referenced_tweets", []
    "mentions": mentions,
    "raw_entities": {
        "hashtags": hashtags,
        "urls": urls,
    },
    "source": "twitter_api",
}
return normalized
```

```yaml
figures:
  # US celebrities / billionaire artists
  - canonical_name: "Taylor Swift"
    patterns: ["taylor swift", "@taylorswift13"]
    type: "person"
    country: "US"
    tags: ["celebrity", "musician", "billionaire"]

  - canonical_name: "Jay-Z"
    patterns: ["jay-z", "jay z", "@sc"]
    type: "person"
    country: "US"
    tags: ["celebrity", "musician", "billionaire", "entr

  - canonical_name: "Rihanna"
    patterns: ["rihanna", "@rihanna"]
    type: "person"
    country: "Barbados"
    tags: ["celebrity", "musician", "billionaire", "entr

  # US tech / finance billionaires
  - canonical_name: "Elon Musk"
    patterns: ["elon musk", "@elonmusk"]
    type: "person"
    country: "US"
    tags: ["billionaire", "tech", "ceo"]

  - canonical_name: "Jeff Bezos"
```

Data Model: Edges

- VADER does analysis on text and categorize the tweet and positive, negative or neutral.

```
>>> df = spark.read.parquet("/data/raw_parquet")
>>> df.printSchema()
root
 |-- src: string (nullable = true)
 |-- dst: string (nullable = true)
 |-- tweet_id: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- sentiment_score: double (nullable = true)
 |-- sentiment_label: string (nullable = true)
```

```
_analyzer = SentimentIntensityAnalyzer()

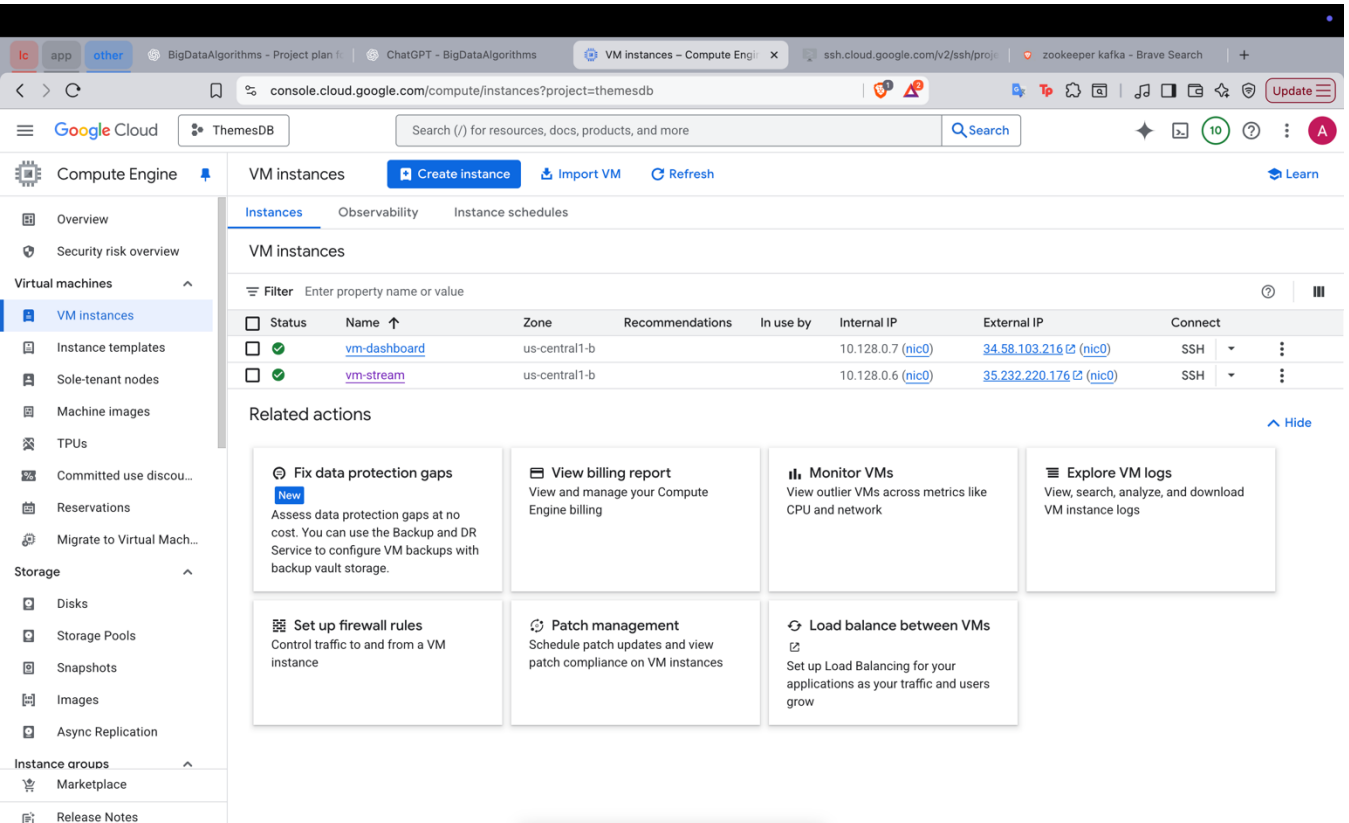

def _score_text(text: str):
    if text is None:
        return 0.0, "neu"
    scores = _analyzer.polarity_scores(text)
    compound = scores.get("compound", 0.0)
    if compound >= 0.05:
        label = "pos"
    elif compound <= -0.05:
        label = "neg"
    else:
        label = "neu"
    return compound, label


def getVaderUDF():
    schema = StructType([
        StructField("compound_score", DoubleType(), nullable=False),
        StructField("label", StringType(), nullable=False),
    ])

    @F.udf(returnType=schema)
    def vader_struct(text):
        c, l = _score_text(text)
        return {"compound_score": c, "label": l}
    return vader_struct
```

Implementation Setup

- Create GCP project, VMs, and GCS bucket
- Install Docker, Python, Java, and set up git repo structure
- Verify Docker plus basic containers run



fra > ! docker-compse.yml

```yaml
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    container_name: kafka
    hostname: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
      - "29092:29092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: "zookeeper:2181"

      # single plaintext listener on 9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEX
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:29092,PLAINTEXT_HOST://0.0.0.
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT

      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

  spark-master:
    image: apache/spark:3.5.1
    container_name: spark-master
```

Ingestion Pipeline: Producer → Kafka

- Have Zookeeper & Kafka up with Docker Compose
- Implement Python producer to:
  - Read from sample tweet file or Twitter's API
  - Normalize and send messages to Kafka topic tweets
- Validate with Kafka CLI to confirm data flow

{"id": "1996818032992424212", "created_at": "2025-12-05T05:44:52.000Z", "text": "@StephenAtHome @ConanOBrien @jonstewart @billmaher @jordanklepper @GabiwilsonX @HERMusicx @HERbrasiI @HerArgentinaOf @GiorgiaMeloni @elonmusk the #AfriKKKaner & @Meta #MarkCukerberg trying to get plugged as the \"unseen guiding hand of Prophecy.\"\nhttps://t.co/uvvGcRJpNl", "author_id": "1767080982749020161", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996817329368584594"}], "mentions": ["StephenAtHome", "ConanOBrien", "jonstewart", "billmaher", "jordanklepper", "GabiwilsonX", "HERMusicx", "HERbrasiI", "HerArgentinaOf", "GiorgiaMeloni", "elonmusk", "Meta"], "raw_entities": {"hashtags": ["AfriKKKaner", "MarkCukerberg"], "urls": ["https://youtu.be/5GkueMtjW4A?si=PFMbQNsB5Y-OqDxH"]}, "source": "twitter_api"}
{"id": "1996818032447181058", "created_at": "2025-12-05T05:44:52.000Z", "text": "@TrashcanElvis @elonmusk @iliketeslas Yes, white gentiles can immigrate to Israel, though not under the Law of Return (reserved for Jews and descendants). Options include naturalization after 3+ years of permanent residency, Hebrew proficiency, and renouncing other citizenships; or via marriage to an Israeli, work", "author_id": "1720665183188922368", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996817802205053423"}], "mentions": ["TrashcanElvis", "elonmusk", "iliketeslas"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996818032145117258", "created_at": "2025-12-05T05:44:52.000Z", "text": "@Crazyflix94 @elonmusk Or replace the population (but slowly).\nRead Agenda 2030!", "author_id": "1797155733743521793", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996778663602483203"}], "mentions": ["Crazyflix94", "elonmusk"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996818032103248261", "created_at": "2025-12-05T05:44:52.000Z", "text": "@elonmusk @DVanLangenhove This is horrific. The poor children being subjected to this disgusting violence. Scarred probably for life. So sad.\ud83d\ude1e", "author_id": "2743155858", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996817608075890854"}], "mentions": ["elonmusk", "DVanLangenhove"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996818031818014960", "created_at": "2025-12-05T05:44:52.000Z", "text": "@NitikaS01339561 @elonmusk @Rudra221098 I am not facing this issue now!", "author_id": "595499436", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996817787491377363"}], "mentions": ["NitikaS01339561", "elonmusk", "Rudra221098"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827479282168026", "created_at": "2025-12-05T06:22:24.000Z", "text": "@AJOnChain7 @MemeCore_M meta\u2019s only getting sharper daily", "author_id": "48635300", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996822000321679705"}], "mentions": ["AJOnChain7", "MemeCore_M"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827479043109347", "created_at": "2025-12-05T06:22:24.000Z", "text": "RT @LaliceUpdates: Leaked report by kmedia - October 15, 2025\n\nTable Read - October 22, 2025 \n\n@wearelloud \n\nLALISA ON NETFLIX\n#LISAxTYGO h\u2026", "author_id": "1585066502981840896", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "retweeted", "id": "1996816323381526725"}], "mentions": ["LaliceUpdates", "wearelloud"], "raw_entities": {"hashtags": ["LISAxTYGO"], "urls": []}, "source": "twitter_api"}
{"id": "1996827478753763786", "created_at": "2025-12-05T06:22:24.000Z", "text": "RT @AppleMusic: \ud83d\udea8 @Wizkidayo and @AsakeMusic announce their new collaboration project, REAL, during their Apple Music Radio takeover. https\u2026", "author_id": "1804473938820345856", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "retweeted", "id": "1996802530656018447"}], "mentions": ["AppleMusic", "wizkidayo", "asakemusic"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478745399319", "created_at": "2025-12-05T06:22:24.000Z", "text": "Amazon\u3067\u3044\u304f\u3089\u8cfc\u5165\u3002\n\u6b63\u76f4\u671f\u5f85\u3057\u3066\u306a\u304b\u3063\u305f\u3051\u3069\u307e\u3058\u3067\u3046\u307e\u3044\u3002\n\u59bb\u3068\u5b50\u3069\u3082\u306e\u53cd\u5fdc\u304c\u5168\u3066\u3060\u3063\u305f\u3002(pr)", "author_id": "1962735913714520064", "in_reply_to_user_id": null, "referenced_tweets": [], "mentions": [], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478678294854", "created_at": "2025-12-05T06:22:24.000Z", "text": "\u6b63\u76f4\u3001Amazon\u3067\u6d77\u9bae\u2026\u3063\u3066\u601d\u3063\u3066\u305f\u3051\u3069\u3001\n\u98df\u3079\u305f\u77ac\u9593\u5bb6\u65cf\u306e\u30c6\u30f3\u30b7\u30e7\u30f3\u7206\u4e0a\u304c\u308a(pr)", "author_id": "1962735913714520064", "in_reply_to_user_id": null, "referenced_tweets": [], "mentions": [], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478678220863", "created_at": "2025-12-05T06:22:24.000Z", "text": "@johnpavlovitz The left is so corrupt that they will change the law, but just for a year, and of course not the one for the actual crime of rape, so that today John could accuse the POTUS of rape.", "author_id": "48515545", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996700194772074553"}], "mentions": ["johnpavlovitz"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478611177565", "created_at": "2025-12-05T06:22:24.000Z", "text": "@elonmusk Buy solana @solana", "author_id": "1466868392602660865", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "replied_to", "id": "1996822877262487713"}], "mentions": ["elonmusk", "solana"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478426534179", "created_at": "2025-12-05T06:22:24.000Z", "text": "RT @mintmag_th: \u0e20\u0e32\u0e1e\u0e22\u0e19\u0e15\u0e23\u0e4c\u0e40\u0e23\u0e37\u0e48\u0e2d\u0e07\u0e41\u0e23\u0e01! #LISA \u0e40\u0e15\u0e23\u0e35\u0e22\u0e21\u0e23\u0e31\u0e1a\u0e1a\u0e17\u0e1a\u0e32\u0e17\u0e43\u0e19 #TYGO \u0e20\u0e32\u0e1e\u0e22\u0e19\u0e15\u0e23\u0e4c\u0e41\u0e2d\u0e47\u0e01\u0e0a\u0e31\u0e48\u0e19\u0e17\u0e23\u0e34\u0e25\u0e40\u0e25\u0e2d\u0e23\u0e4c Netflix \u0e40\u0e23\u0e37\u0e48\u0e2d\u0e07\u0e43\u0e2b\u0e21\u0e48 \u0e08\u0e31\u0e01\u0e23\u0e27\u0e32\u0e25\u0e40\u0e14\u0e35\u0e22\u0e27\u0e01\u0e31\u0e1a \u0e40\u0e23\u0e37\u0e48\u0e2d\u0e07 EXTRACT\u2026", "author_id": "776433090814103552", "in_reply_to_user_id": null, "referenced_tweets": [
{"type": "retweeted", "id": "1996791353506922836"}], "mentions": ["mintmag_th"], "raw_entities": {"hashtags": ["LISA", "TYGO"], "urls": []}, "source": "twitter_api"}
{"id": "1996827478338556241", "created_at": "2025-12-05T06:22:24.000Z", "text": "RT @elonmusk: Tesla self-driving is incredible", "author_id": "337624093", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "retweeted", "id": "1996628365424709695"}], "mentions": ["elonmusk"], "raw_entities": {"hashtags": [], "urls": []}, "source": "twitter_api"}
{"id": "1996827478133035182", "created_at": "2025-12-05T06:22:24.000Z", "text": "RT @test84616661: \u9031\u520a\u30d7\u30ec\u30a4\u30dc\u30fc\u30a4\uff08\u9031\u30d7\u30ec\uff0951\u53f7\n#\u9ad9\u91ce\u771f\u592e\n#\u5c0f\u9ce5\u904a\u308b\u3044 #\u8fd1\u85e4\u6c99\u745b\u5b50 #\u5b87\u54b2 #\u5cb8\u307f\u3086 #\u795e\u5357\u308a\u306a #\u30d0\u30d0\u30d0\u30d0\u30f3\u30d3\n#\u6e1a\u306e\u3093 #\u82b1\u57ce\u5948\u592e #\u90fd\u4e38\u7d17\u4e5f\u83ef #\u5bae\u8d8a\u8679\u6d77 #\u661f\u91ce\u7434\n#\u9031\u30d7\u30ec\nhttps://t.co/9y2CEyaH60 htt\u2026", "author_id": "1056240356755271680", "in_reply_to_user_id": null, "referenced_tweets": [{"type": "retweeted", "id": "1996790885926031484"}], "mentions": ["test84616661"], "raw_entities": {"hashtags": ["\u9ad9\u91ce\u771f\u592e", "\u5c0f\u9ce5\u904a\u308b\u3044", "\u8fd1\u85e4\u6c99\u745b\u5b50", "\u5b87\u54b2", "\u5cb8\u307f\u3086", "\u795e\u5357\u308a\u306a", "\u30d0\u30d0\u30d0\u30d0\u30f3\u30d3", "\u6e1a\u306e\u3093", "\u82b1\u57ce\u5948\u592e", "\u90fd\u4e38\u7d17\u4e5f\u83ef", "\u5bae\u8d8a\u8679\u6d77", "\u661f\u91ce\u7434", "\u9031\u30d7\u30ec"], "urls": ["https://amzn.asia/d/boWjEJS"]}, "source": "twitter_api"}
^CProcessed a total of 525 messages

Streaming Pipeline: Spark Structured Streaming

- Deploy Spark master & workers in Docker
- Implement Spark Structured Streaming job to:
  - Read tweets from Kafka
  - Parse JSON into DataFrame
  - Match entities and compute sentiment
  - Emit raw edges and write to Parquet in GCS with checkpointing
- Confirm that Parquet data and _spark_metadata are correctly produced

```
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ ls -R ~/twitter-bigdata-project/data/raw_parquet
/home/angelcoronel15/twitter-bigdata-project/data/raw_parquet:
_spark_metadata                                                 part-00000-6a488bb3-dfb8-40f5-b003-eea0f7f18753-c000.snappy.parquet
part-00000-086ffeaa-256c-47a4-9973-0095ff73fdd5-c000.snappy.parquet  part-00000-7486ef87-c5f3-438d-8abe-cab14b1cb673-c000.snappy.parquet
part-00000-18a9f8ef-8c05-4d50-ad25-cbb0e6baa450-c000.snappy.parquet  part-00000-85771869-0e06-4a95-b0eb-f4b80efa0ab9-c000.snappy.parquet
part-00000-2f46f37b-7bd4-47d4-9e52-8ea1a66665c8-c000.snappy.parquet  part-00000-c6db04e4-dca9-4081-a867-c6665f49061f-c000.snappy.parquet
part-00000-65ddec9f-496a-4a23-8475-62f501a5b24d-c000.snappy.parquet  part-00000-f9f792fe-0049-4979-9951-a53d0d00f643-c000.snappy.parquet
part-00000-6995b06b-6746-40c0-86db-c28b042c8ea4-c000.snappy.parquet

/home/angelcoronel15/twitter-bigdata-project/data/raw_parquet/_spark_metadata:
0  1  2  3  4  5  6  7  8  9.compact
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ cd ~/twitter-bigdata-project
source venv/bin/activate
python ingestion/producer.py --file data/sample_tweets.jsonl --interval 0.3
Streaming from data/sample_tweets.jsonl to Kafka topic 'tweets'...
 Produced record to tweets partition [0] @ offset 345
 Produced record to tweets partition [0] @ offset 346
 Produced record to tweets partition [0] @ offset 347
 Produced record to tweets partition [0] @ offset 348
 Produced record to tweets partition [0] @ offset 349
 Produced record to tweets partition [0] @ offset 350
 Produced record to tweets partition [0] @ offset 351
 Produced record to tweets partition [0] @ offset 352
 Produced record to tweets partition [0] @ offset 353
 Produced record to tweets partition [0] @ offset 354
 Produced record to tweets partition [0] @ offset 355
 Produced record to tweets partition [0] @ offset 356
 Produced record to tweets partition [0] @ offset 357
 Produced record to tweets partition [0] @ offset 358
 Produced record to tweets partition [0] @ offset 359
 Produced record to tweets partition [0] @ offset 360
 Produced record to tweets partition [0] @ offset 361
 Produced record to tweets partition [0] @ offset 362
 Produced record to tweets partition [0] @ offset 363
 Produced record to tweets partition [0] @ offset 364
 Produced record to tweets partition [0] @ offset 365
 Produced record to tweets partition [0] @ offset 366
 Produced record to tweets partition [0] @ offset 367
 Produced record to tweets partition [0] @ offset 368
 Produced record to tweets partition [0] @ offset 369
 Produced record to tweets partition [0] @ offset 370
 Produced record to tweets partition [0] @ offset 371
 Produced record to tweets partition [0] @ offset 372
 Produced record to tweets partition [0] @ offset 373
 Produced record to tweets partition [0] @ offset 374
 Produced record to tweets partition [0] @ offset 375
 Produced record to tweets partition [0] @ offset 376
 Produced record to tweets partition [0] @ offset 377
 Produced record to tweets partition [0] @ offset 378
 Produced record to tweets partition [0] @ offset 379
 Produced record to tweets partition [0] @ offset 380
 Produced record to tweets partition [0] @ offset 381
 Produced record to tweets partition [0] @ offset 382
```

Batch Analytics (Results)

- Aggregations, Metrics, and Experiments
  - Build batch Spark job for edge stats aggregation
  - Run throughput / latency experiments based on Spark query progress
  - Perform fault tolerance experiment:
    - Kill driver and restart with same checkpoint and verify data

```
>>>
>>> df = spark.read.parquet("/data/raw_parquet")
>>> df.printSchema()
root
 |-- src: string (nullable = true)
 |-- dst: string (nullable = true)
 |-- tweet_id: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- sentiment_score: double (nullable = true)
 |-- sentiment_label: string (nullable = true)

>>> df.show(20, truncate=False)
+-----------+-----------+--------+--------------------+---------------+---------------+
|src        |dst        |tweet_id|timestamp           |sentiment_score|sentiment_label|
+-----------+-----------+--------+--------------------+---------------+---------------+
|Elon Musk  |SpaceX     |1       |2025-11-29T20:15:00Z|0.0            |neu            |
|Elon Musk  |Tesla      |1       |2025-11-29T20:15:00Z|0.0            |neu            |
|SpaceX     |Tesla      |1       |2025-11-29T20:15:00Z|0.0            |neu            |
|Alibaba    |Jack Ma    |2       |2025-11-29T20:16:00Z|0.0            |neu            |
|Elon Musk  |Jeff Bezos |5       |2025-11-29T20:19:00Z|0.0            |neu            |
|Apple      |Tim Cook   |6       |2025-11-29T20:20:00Z|0.296          |pos            |
|Elon Musk  |Tesla      |10      |2025-11-29T20:24:00Z|0.3182         |pos            |
|Bill Gates |Jeff Bezos |12      |2025-11-29T20:26:00Z|0.0            |neu            |
|Alibaba    |Jack Ma    |13      |2025-11-29T20:27:00Z|0.0            |neu            |
|Blue Origin|Jeff Bezos |17      |2025-11-29T20:31:00Z|0.0            |neu            |
|Apple      |Tim Cook   |18      |2025-11-29T20:32:00Z|0.4215         |pos            |
|Elon Musk  |Taylor Swift|21     |2025-11-29T20:35:00Z|0.2023         |pos            |
|Bill Gates |Elon Musk  |24      |2025-11-29T20:38:00Z|0.0516         |pos            |
|Elon Musk  |Jeff Bezos |32      |2025-11-29T20:46:00Z|0.0            |neu            |
|Jay-Z      |Jeff Bezos |35      |2025-11-29T20:49:00Z|0.0            |neu            |
|Jack Ma    |Tim Cook   |40      |2025-11-29T20:54:00Z|0.5423         |pos            |
|Bill Gates |Jeff Bezos |41      |2025-11-29T20:55:00Z|0.0            |neu            |
|Apple      |Elon Musk  |43      |2025-11-29T20:57:00Z|0.25           |pos            |
|Elon Musk  |Jeff Bezos |49      |2025-11-29T21:03:00Z|0.0            |neu            |
|Apple      |Elon Musk  |55      |2025-11-29T21:09:00Z|0.0            |neu            |
+-----------+-----------+--------+--------------------+---------------+---------------+
only showing top 20 rows

>>>
>>> df.show(50, truncate=False)
```

```
NameError: name 'df' is not defined
>>> df = spark.read.parquet("/data/raw_parquet")
>>> edges = df
>>> edge_stats = edges.groupBy("src", "dst").agg(
...     F.count("*").alias("co_mentions"),
...     F.avg("sentiment_score").alias("avg_sentiment")
... )
>>> edge_stats.cache()
DataFrame[src: string, dst: string, co_mentions: bigint, avg_sentiment: double]
>>> edge_stats.show(10, truncate=False)
+-----------+-----------+-----------+--------------------+
|src        |dst        |co_mentions|avg_sentiment       |
+-----------+-----------+-----------+--------------------+
|Elon Musk  |Jay-Z      |6          |0.1806              |
|Amazon     |Jeff Bezos |3          |0.4215              |
|Elon Musk  |SpaceX     |58         |0.0                 |
|Apple      |Tim Cook   |11         |0.26090909090909087 |
|Blue Origin|Jeff Bezos |4          |0.0                 |
|Jeff Bezos |Taylor Swift|3         |0.2023              |
|Jack Ma    |Taylor Swift|3         |0.2023              |
|Elon Musk  |Tesla      |64         |0.026167187499999998|
|Google     |Jeff Bezos |3          |0.0                 |
|Apple      |Elon Musk  |7          |0.14285714285714285 |
+-----------+-----------+-----------+--------------------+
only showing top 10 rows

>>> edge_stats.orderBy(F.desc("co_mentions")).show(10, truncate=False)

+----------+----------+-----------+--------------------+
|src       |dst       |co_mentions|avg_sentiment       |
+----------+----------+-----------+--------------------+
|Elon Musk |Tesla     |64         |0.026167187499999998|
|Alibaba   |Jack Ma   |61         |0.0                 |
|SpaceX    |Tesla     |58         |0.0                 |
|Elon Musk |SpaceX    |58         |0.0                 |
|Elon Musk |Jeff Bezos|17         |0.06737647058823529 |
|Bill Gates|Jeff Bezos|11         |0.09272727272727273 |
|Apple     |Tim Cook  |11         |0.26090909090909087 |
|Bill Gates|Jack Ma   |9          |0.17556666666666665 |
|Bill Gates|Elon Musk |7          |0.1931142857142857  |
|Jack Ma   |Tim Cook  |7          |0.5356142857142857  |
+----------+----------+-----------+--------------------+
only showing top 10 rows

>>>
>>>
```

Experiments: Throughput & Latency

SSH-in-browser UPLOAD FILE DOWNLOAD FILE

```
downloading https://repo1.maven.org/maven2/org/apache/kafka/kafka-clients/3.4.1/kafka-clients-3.4.1.jar ...
        [SUCCESSFUL ] org.apache.kafka#kafka-clients;3.4.1!kafka-clients.jar (248ms)
downloading https://repo1.maven.org/maven2/com/google/code/findbugs/jsr305/3.0.0/jsr305-3.0.0.jar ...
        [SUCCESSFUL ] com.google.code.findbugs#jsr305;3.0.0!jsr305.jar (74ms)
downloading https://repo1.maven.org/maven2/org/apache/commons/commons-pool2/2.11.1/commons-pool2-2.11.1.jar ...
        [SUCCESSFUL ] org.apache.commons#commons-pool2;2.11.1!commons-pool2.jar (102ms)
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-client-runtime/3.3.4/hadoop-client-runtime-3.3.4.jar ...
        [SUCCESSFUL ] org.apache.hadoop#hadoop-client-runtime;3.3.4!hadoop-client-runtime.jar (807ms)
downloading https://repo1.maven.org/maven2/org/lz4/lz4-java/1.8.0/lz4-java-1.8.0.jar ...
        [SUCCESSFUL ] org.lz4#lz4-java;1.8.0!lz4-java.jar (154ms)
downloading https://repo1.maven.org/maven2/org/xerial/snappy/snappy-java/1.1.10.3/snappy-java-1.1.10.3.jar ...
        [SUCCESSFUL ] org.xerial.snappy#snappy-java;1.1.10.3!snappy-java.jar(bundle) (128ms)
downloading https://repo1.maven.org/maven2/org/slf4j/slf4j-api/2.0.7/slf4j-api-2.0.7.jar ...
        [SUCCESSFUL ] org.slf4j#slf4j-api;2.0.7!slf4j-api.jar (101ms)
downloading https://repo1.maven.org/maven2/org/apache/hadoop/hadoop-client-api/3.3.4/hadoop-client-api-3.3.4.jar ...
        [SUCCESSFUL ] org.apache.hadoop#hadoop-client-api;3.3.4!hadoop-client-api.jar (528ms)
downloading https://repo1.maven.org/maven2/commons-logging/commons-logging/1.1.3/commons-logging-1.1.3.jar ...
        [SUCCESSFUL ] commons-logging#commons-logging;1.1.3!commons-logging.jar (85ms)
:: resolution report :: resolve 6833ms :: artifacts dl 2483ms
        :: modules in use:
        com.google.code.findbugs#jsr305;3.0.0 from central in [default]
        commons-logging#commons-logging;1.1.3 from central in [default]
        org.apache.commons#commons-pool2;2.11.1 from central in [default]
        org.apache.hadoop#hadoop-client-api;3.3.4 from central in [default]
        org.apache.hadoop#hadoop-client-runtime;3.3.4 from central in [default]
        org.apache.kafka#kafka-clients;3.4.1 from central in [default]
        org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 from central in [default]
        org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 from central in [default]
        org.lz4#lz4-java;1.8.0 from central in [default]
        org.slf4j#slf4j-api;2.0.7 from central in [default]
        org.xerial.snappy#snappy-java;1.1.10.3 from central in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   11  |   11  |   11  |   0   ||   11  |   11  |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-bf4c9694-4209-4a3c-839e-fc495b3cba01
        confs: [default]
        11 artifacts copied, 0 already retrieved (56767kB/86ms)
25/11/30 19:21:10 INFO SparkContext: Running Spark version 3.5.1
25/11/30 19:21:10 INFO SparkContext: OS info Linux, 6.8.0-1044-gcp, amd64
25/11/30 19:21:10 INFO SparkContext: Java version 11.0.22
25/11/30 19:21:10 INFO ResourceUtils: ==============================================================
25/11/30 19:21:10 INFO ResourceUtils: No custom resources configured for spark.driver.
25/11/30 19:21:10 INFO ResourceUtils: ==============================================================
25/11/30 19:21:10 INFO SparkContext: Submitted application: TwitterGraphStream
25/11/30 19:21:10 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0
, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
25/11/30 19:21:10 INFO ResourceProfile: Limiting resource is cpu
25/11/30 19:21:10 INFO ResourceProfileManager: Added ResourceProfile id: 0
25/11/30 19:21:11 INFO SecurityManager: Changing view acls to: spark
25/11/30 19:21:11 INFO SecurityManager: Changing modify acls to: spark
```

Experiments: Fault Tolerance

SSH-in-browser

UPLOAD FILE DOWNLOAD FILE

```
Produced record to tweets partition [0] @ offset 436
Produced record to tweets partition [0] @ offset 437
Produced record to tweets partition [0] @ offset 438
Produced record to tweets partition [0] @ offset 439
Produced record to tweets partition [0] @ offset 440
Produced record to tweets partition [0] @ offset 441
Produced record to tweets partition [0] @ offset 442
Produced record to tweets partition [0] @ offset 443
Produced record to tweets partition [0] @ offset 444
Produced record to tweets partition [0] @ offset 445
Produced record to tweets partition [0] @ offset 446
Produced record to tweets partition [0] @ offset 447
Produced record to tweets partition [0] @ offset 448
Produced record to tweets partition [0] @ offset 449
Produced record to tweets partition [0] @ offset 450
Produced record to tweets partition [0] @ offset 451
Produced record to tweets partition [0] @ offset 452
Produced record to tweets partition [0] @ offset 453
Produced record to tweets partition [0] @ offset 454
Produced record to tweets partition [0] @ offset 455
Produced record to tweets partition [0] @ offset 456
Produced record to tweets partition [0] @ offset 457
Produced record to tweets partition [0] @ offset 458
Produced record to tweets partition [0] @ offset 459
Produced record to tweets partition [0] @ offset 460
Produced record to tweets partition [0] @ offset 461
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ docker kill spark-master
spark-master
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ docker start spark-master
spark-master
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ docker ps
CONTAINER ID   IMAGE                             COMMAND                  CREATED       STATUS         PORTS
       NAMES
4b9fd3c0811f   apache/spark:3.5.1                "/opt/entrypoint.sh …"   2 hours ago   Up 2 hours     0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
       spark-worker
389dd107d1cf   apache/spark:3.5.1                "/opt/entrypoint.sh …"   2 hours ago   Up 3 seconds   0.0.0.0:7077->7077/tcp, [::]:7077->7077/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tc
p      spark-master
f82aa7617887   confluentinc/cp-kafka:7.6.0       "/etc/confluent/dock…"   2 hours ago   Up 2 hours     0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp, 0.0.0.0:29092->29092/tcp, [::]:29092->2909
2/tcp   kafka
35d54bb91edf   confluentinc/cp-zookeeper:7.6.0   "/etc/confluent/dock…"   2 hours ago   Up 2 hours     2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 3888/tcp
       zookeeper
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$ ls -R ~/twitter-bigdata-project/data/raw_parquet
/home/angelcoronel15/twitter-bigdata-project/data/raw_parquet:
_spark_metadata                                                  part-00000-6995b06b-6746-40c0-86db-c28b042c8ea4-c000.snappy.parquet
part-00000-0062cd2c-22c8-43d8-ae18-856d83385aa3-c000.snappy.parquet  part-00000-6a488bb3-dfb8-40f5-b003-eea0f7f18753-c000.snappy.parquet
part-00000-086ffeaa-256c-47a4-9973-0095ff73fdd5-c000.snappy.parquet  part-00000-7486ef87-c5f3-438d-8abe-cab14b1cb673-c000.snappy.parquet
part-00000-18a9f8ef-8c05-4d50-ad25-cbb0e6baa450-c000.snappy.parquet  part-00000-85771869-0e06-4a95-b0eb-f4b80efa0ab9-c000.snappy.parquet
part-00000-2f46f37b-7bd4-47d4-9e52-8ea1a66665c8-c000.snappy.parquet  part-00000-8774dc4a-8735-4e46-83a1-0d29a2c94fa5-c000.snappy.parquet
part-00000-44924102-b05c-4aef-a54d-ab7e1c520e1c-c000.snappy.parquet  part-00000-c6db04e4-dca9-4081-a867-c6665f49061f-c000.snappy.parquet
part-00000-65ddec9f-496a-4a23-8475-62f501a5b24d-c000.snappy.parquet  part-00000-f9f792fe-0049-4979-9951-a53d0d00f643-c000.snappy.parquet

/home/angelcoronel15/twitter-bigdata-project/data/raw_parquet/_spark_metadata:
0  1  10  11  12  2  3  4  5  6  7  8  9.compact
(venv) angelcoronel15@vm-stream:~/twitter-bigdata-project$
```

Limitations

- Only have access to the free tier of Twitter API which limited the amount of testing I could do
  - Can’t stress test system. Limited to the small test file
- VADER doesn’t understand sarcasm, irony or memes. It only does surface level or “face-value” meaning when going through a tweet
  - Only English is covered. Wanted to add Russian/Chinese entities to project to reach a more “polarized” graph with high neg/pos edges
- Limited resources don’t let me show a “proper” graph of relationships of entities
  - Didn’t present one because mainly its just sparse nodes with few random (biased?) edges

Future Work

- Want to add a UI to visualize data. Possibly live monitoring
- Find other API that will have more flexibility in free tear for streaming
- Can possibly add a catalog that will evolve as new people/companies are added into trends (ex. Luigi Mangione)
- Change VADER for a better model and fine-tune it to match Twitter's type of tweets

Conclusion

What I Built



An end-to-end streaming analytics pipeline on GCP (Kafka/Spark/Parquet)



A relationship graph for key people and companies with weighted sentiment edges



A layer for edge statistics and graph analysis

Takeaways

- Streaming tools make it easy to stream and analyze data
- Even a simple AI can be convenient in finding relationship patterns
- Separating ingestion, processing, storing and analyzing data makes it easier to debug a data pipeline

Q&A

